

CSA0404-OPERATING SYSTEMS

LAB PRACTICE

Name: NAVEEN S

Reg.No:192524030

1. Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

Given:-

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("Process ID: %d\n", getpid());
6     printf("Parent Process ID: %d\n", getppid());
7     return 0;
8 }
```

Process ID: 7191
Parent Process ID: 7190

...Program finished with exit code 0
Press ENTER to exit console.

2. Identify the system calls to copy the content of one file to another and illustrate the same using a C program

Given:-

```
1  #include <stdio.h>
2
3  int main() {
4      FILE *f1, *f2;
5      char ch;
6
7      f1 = fopen("input.txt", "r");
8      f2 = fopen("output.txt", "w");
9
10     while ((ch = fgetc(f1)) != EOF)
11         fputc(ch, f2);
12
13     printf("File copied successfully.");
14
15     fclose(f1);
16     fclose(f2);
17
18     return 0;
19 }
```

Output:-

```
..Program finished with exit code 139  
Press ENTER to exit console.
```

3. Design a CPU scheduling program with C using First Come First Served

```
C:\Users\jaswa\OneDrive\Doc x + v  
Enter the filename to open for reading  
C:/Users/jaswa/OneDrive/Desktop/source.txt  
Enter the filename to open for writing  
C:/Users/jaswa/OneDrive/Desktop/destination  
  
Contents copied to C:/Users/jaswa/OneDrive/Desktop/destination  
-----  
Process exited after 168.3 seconds with return value 0  
Press any key to continue . . .
```

technique with the following considerations.

- a. All processes are activated at time 0.
- b. Assume that no process waits on I/O devices.

AIM: Design a CPU scheduling program with C using First Come First

Served technique with the following considerations.

- a. All processes are activated at time 0.
- b. Assume that no process waits on I/O devices.

Given:-

4. Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next.

Given:-

```
#include<stdio.h>
int main()
{
    int bt[10], wt[10], tat[10], p[10];
    int i, j, n, temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i=0;i<n;i++) {
        printf("P%d Burst Time: ", i+1);
        scanf("%d",&bt[i]);
        p[i]=i+1;
    }

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(bt[i]>bt[j]) {
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=p[i]; p[i]=p[j]; p[j]=temp;
            }

    wt[0]=0;
    for(i=1;i<n;i++)
        wt[i]=wt[i-1]+bt[i-1];

    printf("\nP\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++) {
        tat[i]=bt[i]+wt[i];
        printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);
    }

    return 0;
}
```

Output:-

```

Enter number of processes: 4
P1 Burst Time: 3
P2 Burst Time: 3
P3 Burst Time: 4
P4 Burst Time: 4

P      BT      WT      TAT
P1      3        0        3
P2      3        3        6
P3      4        6       10
P4      4       10       14

...Program finished with exit code 0
Press ENTER to exit console.

```

5. Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

Given:-

```

#include <stdio.h>

int main() {
    int n, i, j, bt[10], pr[10], p[10], wt[10], tat[10], temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &pr[i]);
        p[i] = i + 1;
    }

    for(i = 0; i < n - 1; i++)
        for(j = i + 1; j < n; j++)
            if(pr[i] > pr[j]) {
                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    printf("\nP\tBT\tPR\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", p[i], bt[i], pr[i], wt[i], tat[i]);
    }

    return 0;
}

```

Output:-

Enter number of processes: 3

P1 Burst Time: 9

P1 Priority: 2

P2 Burst Time: 9

P2 Priority: 3

P3 Burst Time: 4

P3 Priority: 2

P	BT	PR	WT	TAT
P1	9	2	0	9
P3	4	2	9	13
P2	9	3	13	22

...Program finished with exit code 0

Press ENTER to exit console.